R Bootcamp for Stats 280 Midterm 2

Learn it, then drill it until you can write it by hand. Built from the R in Lectures 9–15, the review-question solutions, and the W25/W26 Midterm 2 exams.

What the exam asks for

From the Lecture 16–18 slides, Midterm 2 "may ask for R code" to:

1. **Calculate binomial or hypergeometric probabilities**: `dbinom, pbinom, dhyper, phyper` (W25 Q4, Q6; W26 Q4, Q6).
2. **Simulate random variables**: write a function with `runif` or `sample` (W25 Q5a, W26 Q5c).
3. **Estimate probability, expected value, or standard error by simulation**: loops, `mean, sd` (W25 Q5c, W26 Q5d).
4. **Use the normal approximation**: `pnorm` (Lecture 15).

R questions are about 25–35 of the 200 points, and they need no justification. You either write the line correctly or you don't. They're the cheapest points on the exam once the syntax is automatic.

How to use this guide

1. **Learn** each module: read it and run every example in RStudio. Change one thing in each example and predict the result before running it.
2. **Drill**: do every drill *on paper first*. The exam is handwritten, with no autocomplete and no error messages. Then type it in and check. Mark each drill ✓ or ×.
3. **Check** against the separate Answer Key PDF. Every answer in it was run in R 4.3, so the outputs are real.
4. **Redo every ×** the next day without looking. A drill only counts once you get it right cold.

Drill type	What it trains
PREDICT	Read code and write its exact output. Builds a mental model of R's behavior.
WRITE	Write code from an English spec. This is what the exam does.
TRANSLATE	Turn a word problem into one R command. Pure exam format.
FILL	Fill in the blanks in given code. The W25 Q5 format.
DEBUG	Find every bug. Each bug is one exams commonly punish.
EXPLAIN	Say what the code estimates. The W26 Q5d format.
RECALL	Flashcards for pure memorization (Module 8).

Stars mark difficulty: ★ warm-up, ★★ exam level, ★★★ harder than the exam. If you can do the ★★★ drills, the exam's R questions will feel easy.

Suggested schedule

Session	Modules	Time
1	0–3: Python → R, logical vectors, functions and loops	~2 h
2	4–5: exact probabilities (the off-by-one table), randomness	~1.5 h
3	6: simulation recipes, the core of the exam's R	~1.5 h
4	7–8: timed Gauntlet (35 min), then flashcards until perfect	~1 h

Then take the staff's W26 Midterm 2, then Practice Midterms 2A, 2B and 2C.

MODULE 0

Setup and the R console

Goal: run R and read its output format · **Time:** about 10 min

Use RStudio from lab, or `posit.cloud` in a browser. Type code in the console or in a script (Ctrl/Cmd+Enter runs a line).

Output format: R prints `[1]` before a printed vector. The number in brackets is the index of the first element on that line. In this guide, output lines start with `##`, as in the lecture slides.

```
5 + 3
1:30
```

```
## [1] 8
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30
```

> **Rule:** at the top level, a bare expression prints automatically. Inside a `for` loop or a function it does *not*; there you need `print()` or `cat()`. That's why the lecture code says `print(mean(X))`.

MODULE 1

Python → R: the rewiring

Goal: stop writing Python in R · **Time:** about 35 min

You already know how to program. You need to unlearn about a dozen Python reflexes. Here they are with examples.

1. Assignment and vectors

`x = 5` and `x <- 5` do the same thing. The course uses `=`. **Everything is a vector**: `5` is a vector of length 1. `c()` ("combine") builds vectors, and `length()` gives their length.

```
x = c(4, 8, 15, 16, 23, 42)
length(x)
y = c(x, 99)     # c() also appends
y
```

```
## [1] 6
## [1]  4  8 15 16 23 42 99
```

2. Indexing starts at 1, ranges include both ends

```
x[1]            # first element (not x[0])
x[length(x)]    # last element
x[c(1, 3)]      # several positions at once
x[-1]           # NEGATIVE index = drop that element (not "last")
x[2:4]          # 2:4 is 2,3,4 (inclusive!)
x[0]            # no error: an empty vector
x[10]           # no error: NA
```

```
## [1] 4
## [1] 42
## [1]  4 15
## [1]  8 15 16 23 42
## [1]  8 15 16
## numeric(0)
## [1] NA
```

> **Trap:** `x[-1]` means "everything except the first", not "the last". Python's `x[-1]` is R's `x[length(x)]`.

3. Vectorized arithmetic and recycling

Arithmetic works elementwise on whole vectors, with no loop. If the lengths differ, R recycles the shorter vector.

```
c(1, 2, 3) * 2
c(1, 2, 3) + c(10, 20, 30)
(1:6)^2
1:6 + c(0, 100)       # c(0,100) is recycled: 0,100,0,100,0,100
```

```
## [1] 2 4 6
## [1] 11 22 33
## [1]  1  4  9 16 25 36
## [1]   1 102   3 104   5 106
```

4. Operators that differ from Python

Python	R	Note
`2**3`	`2^3`	`**` happens to work, but write `^`
`7 % 3`	`7 %% 3`	modulo
`7 // 3`	`7 %/% 3`	integer division
`True, False`	`TRUE, FALSE`	capitals; `T`/`F` also work
`and, or, not`	`&, |, !`	vectorized (Module 2)
`None` / `[]`	`NULL` / `c()`	empty vector
`range(1, 6)`	`1:5`	R includes the end
`len(x)`	`length(x)`	
`x.append(v)`	`x = c(x, v)`	returns a new vector
`math.comb(n, k)`	`choose(n, k)`	Module 4

5. The colon has HIGH precedence

```
n = 4
1:n-1        # this is (1:n) - 1, NOT 1:(n-1)
1:(n-1)
-1:2         # (-1):2
2*1:3        # 2*(1:3)
```

```
## [1] 0 1 2 3
## [1] 1 2 3
## [1] -1  0  1  2
## [1] 2 4 6
```

> **Trap:** `1:n-1` gives `0 1 2 3`. When in doubt, add parentheses. The W25 key writes `1:(n0+1)`.

6. Building vectors: seq and rep

`rep` is how you build a **box** of tickets. You'll use it constantly.

```
seq(0, 1, 0.25)              # from, to, by
seq(10, 50, by = 10)
rep(7, 3)
rep(c(1, 2), 3)              # times = 3: whole vector repeated
rep(c(1, 2), each = 3)       # each element repeated
rep(c(0, 1), c(39, 13))      # 39 zeros then 13 ones: a deck as non-spade/spade
box = c(1, 1, 2, 4)          # W26 Q5 box, typed directly
```

```
## [1] 0.00 0.25 0.50 0.75 1.00
## [1] 10 20 30 40 50
## [1] 7 7 7
## [1] 1 2 1 2 1 2
## [1] 1 1 1 2 2 2
##  [1] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [39] 0 1 1 1 1 1 1 1 1 1 1 1 1
```

7. The built-ins you need

```
v = c(3, 1, 4, 1, 5, 9, 2, 6)
sum(v); prod(1:5); mean(v); max(v); min(v)
sqrt(16); abs(-3); round(3.14159, 2)
sort(v); rev(1:4); cumsum(1:5)
table(c(2, 5, 5, 7, 7, 7))   # counts of each value
```

```
## [1] 31
## [1] 120
## [1] 3.875
## [1] 9
## [1] 1
## [1] 4
## [1] 3
## [1] 3.14
## [1] 1 1 2 3 4 5 6 9
## [1] 4 3 2 1
## [1]  1  3  6 10 15
##
## 2 5 7
## 1 2 3
```

> A `;` puts several statements on one line. Each one still prints.

Drills

1.1 ★ PREDICT

Write the exact output of each line.

```
x = c(10, 20, 30, 40, 50)
x[2]
x[-2]
x[c(1, 5)]
x[length(x) - 1]
x[2:3] * 2
```

1.2 ★ PREDICT

Precedence. Write each output.

```
n = 5
1:n-1
1:(n-1)
2*1:3
0:4/2
```

1.3 ★ PREDICT

Write each output.

```
10 %% 3
10 %/% 3
2^10
rep(c(1, 2), 3)
rep(c(1, 2), each = 3)
rep(c(0, 1), c(2, 3))
seq(2, 11, 3)
```

1.4 ★ PREDICT

Write each output.

```
sum(1:100)
cumsum(1:5)
prod(5:1)
c(1, 2) + c(10, 20, 30, 40)
length(c(c(1, 2), 3:5))
```

1.5 ★ WRITE

In **one expression each**, create: (a) the box {1, 1, 2, 4}; (b) a box with 30,000 zeros and 10,000 ones (the green M&M box from Review 15 Q5); (c) the numbers 0.05, 0.10, …, 1.00; (d) a vector of the 52 cards' suits, coded 1–4 with 13 of each.

1.6 ★★ WRITE

Compute $\sum_{k=1}^{10} k^2$ and $1 + \frac{1}{2} + \frac{1}{4} + \cdots + \frac{1}{2^{10}}$, each in one line with no loop.

1.7 ★★ WRITE

A box has tickets 2, 5, 5, 7, 7, 7, 9 (Lecture 12). Store it in `box` using `rep`. Then compute its average and its SD *as defined in class* (divide by n, not $n-1$), using only `mean`, `sqrt` and arithmetic.

1.8 ★★★ PREDICT

Tricky. Predict each output, then check. Explain each surprise in one line.

```
x = c(5, 6, 7)
x[c(TRUE, FALSE, TRUE)]
x[-c(1, 2)]
x[4] = 8; x
y = c(); y[3] = 1; y
1:0
```

MODULE 2

Logical vectors: the simulation engine

Goal: count, filter and estimate with TRUE/FALSE · **Time:** about 35 min

Almost every simulation answer ends in `mean(` *some logical vector* `)`. Master this module and the simulation questions become easy.

Comparisons return logical vectors

```
x = c(2, 5, 5, 7, 7, 7, 9)
x > 5
x == 7          # == compares; = assigns
x != 7
```

```
## [1] FALSE FALSE FALSE  TRUE  TRUE  TRUE  TRUE
## [1] FALSE FALSE FALSE  TRUE  TRUE  TRUE FALSE
## [1]  TRUE  TRUE  TRUE FALSE FALSE FALSE  TRUE
```

TRUE counts as 1, FALSE as 0

So `sum` counts and `mean` gives a proportion. If the vector holds simulated outcomes, `mean(event)` **estimates the probability of the event**.

```
sum(x > 5)      # how many are > 5
mean(x == 7)    # fraction equal to 7 (here 3/7)
TRUE + TRUE
```

```
## [1] 4
## [1] 0.4285714
## [1] 2
```

Filtering with a logical index

```
x[x > 5]            # keep elements where condition is TRUE
x[x != 7]
which(x == 7)       # POSITIONS where TRUE
any(x > 8); all(x > 1)
7 %in% x            # membership test
```

```
## [1] 7 7 7 9
## [1] 2 5 5 9
## [1] 4 5 6
## [1] TRUE
## [1] TRUE
## [1] TRUE
```

Combining conditions: &, |, !

```
(x > 2) & (x < 9)
(x == 2) | (x == 9)
!(x == 7)
```

```
## [1] FALSE  TRUE  TRUE  TRUE  TRUE  TRUE FALSE
## [1]  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE
## [1]  TRUE  TRUE  TRUE FALSE FALSE FALSE  TRUE
```

> **Trap:** `&` and `|` are elementwise. `&&` and `||` use only single values and belong inside `if ( )`. In this course, plain `&` works everywhere you need it.

ifelse: a vectorized if

```
ifelse(x > 5, 1, 0)
ifelse(c(1, 0, 1) == 1, "H", "T")
```

```
## [1] 0 0 0 1 1 1 1
## [1] "H" "T" "H"
```

The lecture's "indicator trick"

Lecture 12 turns uniform random numbers into a random variable by multiplying each value by a logical (0/1) condition and adding the pieces. Exactly one condition is TRUE for each `p`, so exactly one term survives.

```
p = c(0.05, 0.30, 0.50, 0.90)    # pretend these came from runif(4)
2*(p < 1/7) + 5*(p >= 1/7 & p < 3/7) + 7*(p >= 3/7 & p < 6/7) + 9*(p >= 6/7)
```

```
## [1] 2 5 7 9
```

Each interval's width is the probability of its value: 2 has width 1/7, 5 has width 2/7, and so on. That's the box {2, 5, 5, 7, 7, 7, 9}.

Conditional proportions: filter, then average

To estimate P(A | B) from simulations, keep only the runs where B happened, then take the proportion where A happened: `mean(A[B])`. Lecture 10's frequency interpretation says exactly this: throw out the runs where B didn't happen.

```
heads  = c(1, 0, 1, 1, 0, 1)        # 6 simulated runs
total  = c(6, 6, 3, 6, 9, 2)
mean(heads[total == 6])             # among runs with total 6, fraction with heads
```

```
## [1] 0.6666667
```

Drills

2.1 ★ PREDICT

Write each output.

```
v = c(3, 8, 1, 8, 6)
v >= 6
sum(v >= 6)
mean(v == 8)
v[v < 5]
which(v == 8)
```

2.2 ★ PREDICT

Write each output.

```
v = c(3, 8, 1, 8, 6)
any(v == 2); all(v > 0)
c(2, 8) %in% v
ifelse(v %% 2 == 0, "even", "odd")
sum(v[v > 2])
```

2.3 ★★ PREDICT

Indicator trick. What does each line print? Which box does the last line simulate?

```
u = c(0.10, 0.25, 0.60, 0.80)
0*(u < 1/4) + 4*(u >= 1/4 & u < 3/4) + 8*(u >= 3/4)
1*(u <= 0.5)
```

2.4 ★ WRITE

In one line each, with no loop: (a) count the integers from 1 to 1000 divisible by 3 or by 5; (b) the fraction of die faces that are at least 5; (c) the sum of the even numbers from 1 to 100.

2.5 ★★ WRITE

`rolls` holds simulated die rolls and `coin` holds matching coin flips (1 = heads), one pair per run. Write one expression for each estimate: (a) P(roll = 6); (b) P(roll is even and coin is heads); (c) P(roll ≥ 5 | coin is heads); (d) the average roll among runs where the coin was tails.

2.6 ★★ WRITE

Using a vector `u` of uniform numbers, write one expression that turns each element into a draw from the box {1, 2, 2, 3} (Lecture 13), using the indicator trick.

2.7 ★★★ DEBUG

Each line is meant to estimate P(sum = 7 | first die = 3) from vectors `d1` and `d2` of simulated rolls. Which lines are correct, and what do the wrong ones estimate?

```
mean(d1 + d2 == 7 & d1 == 3)          # (a)
mean((d1 + d2)[d1 == 3] == 7)          # (b)
sum(d1 + d2 == 7 & d1 == 3) / sum(d1 == 3)   # (c)
mean(d1 + d2 == 7) / mean(d1 == 3)     # (d)
```

MODULE 3

Functions, if/else, loops

Goal: write the functions and loops the exam asks for · **Time:** about 40 min

Functions

```
square = function(x) {
  return(x^2)
}
square(4)
square(1:3)        # vectorized for free

add = function(a, b = 10) {   # default argument
  a + b                       # last expression is returned
}
add(1); add(1, 2)
```

```
## [1] 16
## [1] 1 4 9
## [1] 11
## [1] 3
```

> **Trap:** `return` is a function, so you must write `return(x)`. `return x` is a syntax error. Write explicit `return( )` on the exam; the course solutions do.

if / else

```
sign_of = function(x) {
  if (x > 0) {
    return("positive")
  } else if (x == 0) {
    return("zero")
  } else {
    return("negative")
  }
}
sign_of(-2); sign_of(0)
```

```
## [1] "negative"
## [1] "zero"
```

> **Rule:** always write `} else {` on one line. At the top level, an `else` that starts a new line is a syntax error. Inside a function it happens to work, but don't rely on it.

for loops

```
total = 0
for (i in 1:4) {
  total = total + i^2
}
total

for (v in c(10, 20, 30)) {   # loop directly over values
  print(v / 10)              # inside a loop you need print()
}
```

```
## [1] 30
## [1] 1
## [1] 2
## [1] 3
```

while loops

```
n = 0
total = 0
while (total < 10) {
  n = n + 1
  total = total + n
}
c(n, total)
```

```
## [1]  4 10
```

Two ways to collect results (both appear in course code)

```
# Style 1: start empty and append (Lecture 14, W26 Q5d)
res = c()
for (i in 1:5) {
  res = c(res, i^2)
}
res

# Style 2: preallocate, then fill by index (W25 Q5c)
res2 = numeric(5)        # five 0s
for (i in 1:5) {
  res2[i] = i^2
}
res2
```

```
## [1]  1  4  9 16 25
## [1]  1  4  9 16 25
```

Style 2 is faster for a million runs. Either one is fine on paper. The W25 exam wrote `numeric(1,000,000)`. In real R that's an error, because the commas make three arguments. Write `numeric(1000000)` or `numeric(1e6)`.

sapply and replicate (shortcuts)

```
sapply(1:5, square)                    # apply a function to each element
set.seed(1)
replicate(5, sum(sample(1:6, 2, replace = TRUE)))   # repeat an expression 5 times
```

```
## [1]  1  4  9 16 25
## [1] 5 3 8 8 6
```

`replicate(N, expr)` is the cleanest way to write a simulation. The course code mostly uses `for` loops, so be fluent in both. Use a loop on the exam unless the question invites otherwise.

> **Trap (Python people):** `for (i in 1:n)` with `n = 0` runs *twice* (i = 1, then 0), because `1:0` is `c(1, 0)`. `seq_len(n)` is the safe version. On the exam it only matters if a count can be 0.

Drills

3.1 ★ PREDICT

Write each output.

```
f = function(x) {
  if (x > 3) {
    return(x * 2)
  } else {
    return(x + 1)
  }
}
f(2)
f(5)
sapply(1:5, f)
```

3.2 ★ PREDICT

Write the final value of each variable.

```
s = 0
for (k in 2:5) { s = s + k }
s
p = 1
for (k in c(2, 3, 4)) { p = p * k }
p
v = c()
for (k in 1:3) { v = c(v, k, -k) }
v
```

3.3 ★★ PREDICT

Trace the loop. What prints?

```
count = 0
x = 100
while (x > 1) {
  x = x / 2
  count = count + 1
}
c(count, x)
```

3.4 ★★ WRITE

Write `cointoss(p)` that returns 1 with probability `p` and 0 otherwise, using `runif`. This is W25 Q5a, worth 10 points.

3.5 ★★ WRITE

Write `MAXABS(v)`, which returns the largest absolute value in a vector *using a loop* (Review 7 Q4), with no `max` or `abs`. Test it on 2, 9, 5, −4, 0 and on −12, 3.

3.6 ★★ WRITE

Write `num_heads(n)`, which simulates `n` fair coin tosses and returns the number of heads. Write it twice: (a) with a `for` loop and `cointoss` from 3.4; (b) in one line with no loop.

3.7 ★★ DEBUG

This is meant to return the number of rolls needed to get the first six. Find all **four** bugs.

```
first_six = function() {
  n = 0
  roll = 0
  while (roll = 6) {
    roll = sample(1:6)
    n + 1
  }
  return n
}
```

3.8 ★★★ WRITE

Write `runs_until(k)`: toss a fair coin until you get `k` heads *in a row*, and return the total number of tosses. Use it to estimate the expected number of tosses for k = 2 (exact answer: 6).

3.9 ★★★ PREDICT

What does `g(5)` return, and what does it compute in general?

```
g = function(n) {
  if (n <= 1) {
    return(1)
  }
  return(n * g(n - 1))
}
g(5)
```

MODULE 4

Exact probabilities: counting, binomial, hypergeometric, normal

Goal: turn any counting word problem into one correct line · **Time:** about 50 min

Counting (Lecture 11)

```
factorial(5)                      # 5! orderings of 5 objects
factorial(5) / factorial(5 - 2)  # permutations: ordered choices of 2 from 5
choose(5, 2)                      # combinations: subsets of size 2 from 5
choose(52, 5)                     # 5-card hands
exp(lfactorial(100) - lfactorial(98))   # 100*99 without overflow (lecture tip)
```

```
## [1] 120
## [1] 20
## [1] 10
## [1] 2598960
## [1] 9900
```

Binomial: draws WITH replacement / independent trials

X = number of successes in n independent trials, each with success probability p. Then $P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$.

```
dbinom(2, 5, 0.7)          # P(X = 2):  d = "density" = exactly
pbinom(4, 5, 0.7)          # P(X <= 4): p = cumulative = AT MOST
dbinom(0:5, 5, 0.7)        # all six probabilities at once
sum(dbinom(0:5, 5, 0.7))
```

```
## [1] 0.1323
## [1] 0.83193
## [1] 0.00243 0.02835 0.13230 0.30870 0.36015 0.16807
## [1] 1
```

Hypergeometric: draws WITHOUT replacement

Box with **m** success tickets and **n** failure tickets. Draw **k** without replacement. $P(X = x) = \binom{m}{x}\binom{n}{k-x} / \binom{m+n}{k}$.

> **Memorize the argument order:** `dhyper(x, m, n, k)` = `dhyper(` successes drawn `,` success tickets `,` failure tickets `,` number drawn `)`.
> Note `n` is the number of *failure* tickets, **not** the box size. Mnemonic: **"x of m, from m+n, take k."**

```
# W26 Q4: 12 new + 8 used batteries, take 5 at once, P(exactly 2 new)
dhyper(2, 12, 8, 5)
choose(12, 2) * choose(8, 3) / choose(20, 5)    # same thing by formula
# Lecture 12: 25 smokers, 50 non-smokers, sample 25: P(at most 5 smokers)
phyper(5, 25, 50, 25)
```

```
## [1] 0.2383901
## [1] 0.2383901
## [1] 0.06836793
```

THE off-by-one table (most exam R points depend on this)

`pbinom(k, ...)` and `phyper(k, ...)` give P(X ≤ k), **including k**. Translate every phrase to "≤ something" first.

English	Math	R (binomial; hyper is identical with phyper)
exactly k	$P(X = k)$	`dbinom(k, n, p)`
at most k / k or fewer / no more than k	$P(X \leq k)$	`pbinom(k, n, p)`
fewer than k / less than k	$P(X \leq k-1)$	`pbinom(k-1, n, p)`
at least k / k or more	$1 - P(X \leq k-1)$	`1 - pbinom(k-1, n, p)`
more than k / greater than k	$1 - P(X \leq k)$	`1 - pbinom(k, n, p)`
between a and b inclusive	$P(X \leq b) - P(X \leq a-1)$	`pbinom(b, n, p) - pbinom(a-1, n, p)`
(any of the above, safest)	sum the exact terms	`sum(dbinom(a:b, n, p))`

```
# W25 Q6b: P(Celtics win 60 or more of 82), p = 0.78
1 - pbinom(59, 82, 0.78)
sum(dbinom(60:82, 82, 0.78))       # identical, and impossible to off-by-one
```

```
## [1] 0.8811662
## [1] 0.8811662
```

> **Exam tip:** the official keys use the `pbinom` forms. `sum(dbinom(a:b, ...))` is correct R and gives the same number, so it should earn full credit. If you're unsure of the boundary, it's the safer form.

The normal curve: pnorm and qnorm

```
pnorm(1.96)                 # area to the LEFT of z = 1.96
pnorm(2) - pnorm(-1)        # area between -1 and 2 (Lecture 4)
1 - pnorm(75, 70, 5)        # P(X > 75) when X is normal(mean 70, SD 5)
qnorm(0.90)                 # z for the 90th percentile
qnorm(0.90, 70, 5)          # 90th percentile of normal(70, 5)
```

```
## [1] 0.9750021
## [1] 0.8185946
## [1] 0.1586553
## [1] 1.281552
## [1] 76.40776
```

Normal approximation with continuity correction (Lecture 15)

For a count X (binomial) with $EV = np$ and $SE = \sqrt{np(1-p)}$: to approximate P(a ≤ X ≤ b), use the interval from $a - 0.5$ to $b + 0.5$.

```
# W26 Q6: 150 draws, p = 0.5, P(70 <= reds <= 100)
pbinom(100, 150, 0.5) - pbinom(69, 150, 0.5)                    # exact
ev = 150 * 0.5; se = sqrt(150 * 0.5 * 0.5)
pnorm(100.5, ev, se) - pnorm(69.5, ev, se)                      # normal approx
pnorm((100.5 - ev) / se) - pnorm((69.5 - ev) / se)              # same, via z
```

```
## [1] 0.8154061
## [1] 0.8154306
## [1] 0.8154306
```

Drills

4.1 ★ TRANSLATE

P(exactly 3 heads in 10 tosses of a fair coin). One R command.

4.2 ★ TRANSLATE

P(at least 7 heads in 10 tosses). One R command.

4.3 ★ TRANSLATE

P(more than 7 heads in 10 tosses). One R command.

4.4 ★ TRANSLATE

P(fewer than 2 sixes in 12 rolls of a die). One R command.

4.5 ★★ TRANSLATE

A free-throw shooter makes 65%. P(she makes between 60 and 70 of 100, inclusive). One R command.

4.6 ★ TRANSLATE

Deal 5 cards. P(exactly 2 hearts). One R command.

4.7 ★★ TRANSLATE

Deal 5 cards. P(at least one ace). One R command.

4.8 ★★ TRANSLATE

Committee of 6 chosen at random from 8 women and 10 men. P(at most 2 women). One R command.

4.9 ★★ TRANSLATE

A lot of 100 chips has 8 defective. Sample 10 without replacement. P(more than 3 defective). One R command.

4.10 ★★ TRANSLATE

Same lot. P(between 1 and 3 defective, inclusive). One R command.

4.11 ★★ TRANSLATE

Draw 10 of 15 volunteers (5 men, 10 women) without replacement. P(at least 5 women). Write it two ways: in terms of women and in terms of men. What is the value, and why? One R command.

4.12 ★ TRANSLATE

The number of ways to arrange 4 A's and 7 B's in a row (Review 12 Q5). One R command.

4.13 ★★ TRANSLATE

The number of ways to arrange 4 A's, 7 B's and 3 C's (Review 12 Q6). One R command.

4.14 ★ TRANSLATE

The number of ordered podiums (gold, silver, bronze) from 8 runners. One R command.

4.15 ★ TRANSLATE

Z is standard normal. P(-1 < Z < 2). One R command.

4.16 ★★ TRANSLATE

Heights are normal with mean 64, SD 3. The 80th percentile, and P(height > 70). One R command.

4.17 ★★ TRANSLATE

X is binomial(100, 0.5). Normal approximation, with continuity correction, to P(X ≥ 60). One R command.

4.18 ★★ TRANSLATE

A town of 1,000 has 350 Democrats. A random sample of 100 people, without replacement. P(exactly 35 Democrats). One R command.

4.19 ★★★ TRANSLATE

Roll a die repeatedly. P(the 3rd six occurs exactly on roll 10). (Hint: what must happen in the first 9 rolls?) One R command.

4.20 ★★★ TRANSLATE

P(at least 2 sixes in 7 rolls | at least 1 six in 7 rolls) (Review 12 Q11, adapted). One R command.

4.21 ★★ DEBUG

Each command has an error. Say what's wrong and fix it.

(a) "P(Celtics win 60 or more of 82)": `1 - pbinom(60, 82, 0.78)`

(b) "5 cards, P(exactly 2 hearts)": `dhyper(2, 13, 52, 5)`

(c) "P(at most 3 in binomial(10, 0.4))": `dbinom(3, 10, 0.4)`

(d) "12 new, 8 used, take 5 at once, P(2 new)": `dbinom(2, 5, 12/20)`

(e) "P(X ≥ 60), normal approximation, EV 50, SE 5": `pnorm(59.5, 50, 5)`

4.22 ★★ PREDICT

Predict each output, roughly or exactly.

```
dbinom(0:3, 3, 0.5)
pbinom(3, 3, 0.5)
dhyper(1, 1, 9, 3)
choose(5, 0) + choose(5, 5)
```

4.23 ★★★ WRITE

W25 Q1e asks: drawing cards without replacement until the first spade, what's the smallest number of cards *c* with P(need ≤ c cards) ≥ 90%? Compute P(more than c cards) = P(the first c cards are all non-spades) for c = 1, …, 10 in **one vectorized line**, then pick out the answer.

MODULE 5

Randomness: runif, sample, set.seed

Goal: generate any random experiment in one line · **Time:** about 30 min

runif: uniform numbers between 0 and 1

```
set.seed(280)            # makes "random" output reproducible
runif(3)
u = runif(1)
u < 0.3                  # TRUE with probability 0.3: an event of probability 0.3
```

```
## [1] 0.3787605 0.5164143 0.4490346
## [1] FALSE
```

sample: draw from a vector

`sample(x, size, replace = FALSE, prob = NULL)`. **The default is WITHOUT replacement.**

```
set.seed(1)
sample(1:6, 1)                           # one die roll
sample(1:6, 10, replace = TRUE)          # ten die rolls
sample(c(1, 1, 2, 4), 3)                 # 3 tickets without replacement (W26
box)
sample(c(0, 1), 3, replace = TRUE)       # 3 fair coin tosses as 0/1
sample(c(1, 0), 5, replace = TRUE, prob = c(0.3, 0.7))   # biased coin
sample(1:5)                              # a random shuffle (permutation)
```

```
## [1] 1
##  [1] 4 1 2 5 3 6 2 3 3 1
## [1] 1 4 1
## [1] 1 1 1
## [1] 1 1 0 0 0
## [1] 1 5 2 4 3
```

Trap 1: `sample(1:6, 10)` is an **error**: you can't take 10 without replacement from 6.

Trap 2: `sample(x, 1)` when `x` is a *single number* like 10 samples from `1:10`, not the value 10. This only bites if the vector you sample from can shrink to length 1.

Trap 3: `runif(p)` is not "a uniform with probability p". It makes `p` numbers. Use `runif(1) < p`.

Common building blocks

```
set.seed(2)
sum(sample(1:6, 2, replace = TRUE))       # sum of two dice
deck = rep(c("S", "H", "D", "C"), each = 13)
hand = sample(deck, 5); hand; sum(hand == "H")   # 5-card hand, number of hearts
max(sample(c(1, 1, 2, 4), 2))             # max of 2 tickets, no replacement
rbinom(5, 10, 0.5)                        # 5 draws of binomial(10, 0.5): # heads
in 10 tosses
```

```
## [1] 11
## [1] "S" "D" "S" "H" "D"
## [1] 1
## [1] 4
## [1] 4 6 4 5 7
```

Drills

5.1 ★ WRITE

One line each: (a) roll two dice and return the sum; (b) toss a coin with P(heads) = 0.3 once, returning 1 or 0, *two ways*; (c) draw one ticket from the box {2, 5, 5, 7, 7, 7, 9}; (d) toss a fair coin 3 times and return the number of heads (W26 Q5c).

5.2 ★★ PREDICT

Which of these outputs are **possible** from `sample(c(1, 1, 2, 4), 3)`? (i) `1 1 2` (ii) `4 4 1` (iii) `2 1 1` (iv) `1 2 3` (v) `4 1`

5.3 ★★ WRITE

Simulate drawing 5 cards from a deck and return `TRUE` if the hand is a flush (all 5 the same suit). Hint: `length(unique(v))` counts distinct values. (The exact probability is about 0.00198.)

5.4 ★★ DEBUG

Each line was supposed to do what its comment says. Fix each one.

```
sample(1:6, 10)                          # ten die rolls
runif(0.3) < 1                            # an event with probability 0.3
sample(c(0, 1), 3, prob = c(0.5, 0.5))   # three fair coin tosses
sample(deck, 5, replace = TRUE)          # a 5-card poker hand
```

5.5 ★★★ WRITE

Write `sample_x(n)` that returns `n` independent draws from the box $\{0, 4, 4, 8\}$ **without** using `sample`, in the Lecture 13 style (`runif` plus the indicator trick). Then check its mean and SE against the exact values 4 and $2\sqrt{2} \approx 2.83$.

MODULE 6

Simulation recipes: the core of the exam's R

Goal: estimate any probability, EV or SE by simulation · **Time:** about 60 min

Every simulation question uses the same skeleton: **(1) write a function that runs the experiment once. (2) Run it N times and store the results. (3) Summarize.**

To estimate…	Summarize the stored results with
a probability P(A)	`mean(results satisfy A)`, e.g. `mean(sims > 5)`
an expected value E(X)	`mean(sims)`
a standard error SE(X)	`sd(sims) * sqrt((N - 1) / N)`
a conditional probability P(A | B)	`mean(A[B])`: keep runs where B happened, then take the fraction with A
a covariance Cov(X, Y)	`cov(xs, ys) * (N - 1) / N`
a whole distribution	`table(sims) / N` or `hist(sims, freq = FALSE, breaks = ...)`

> **Why `sqrt((N-1)/N)`?** R's `sd()` divides by $N-1$ (it computes SD^{+}). The class SD divides by N, so `sd(x) * sqrt((N-1)/N)` converts it. Lectures 13–14 write `sd(X)*sqrt(99999999/100000000)`. The W26 key writes `sd(ms)*9999/10000` and calls it the SE estimate; the exact conversion is the square root, but with $N = 10{,}000$ the difference is negligible. Writing `sd(sims)` alone is also a reasonable estimate for large N.

Recipe 1 & 2: one-run function, N runs, summarize (W26 Q5)

```
simM = function() {
  numDraws = sum(sample(c(0, 1), 3, replace = TRUE)) + 1   # heads in 3 tosses,
plus 1
  tickets = sample(c(1, 1, 2, 4), numDraws, replace = FALSE)
  return(max(tickets))
}
set.seed(6)
N = 100000
ms = numeric(N)
for (i in 1:N) {
  ms[i] = simM()
}
mean(ms)                          # E(M); exact 25/8 = 3.125
sd(ms) * sqrt((N - 1) / N)        # SE(M)
table(ms) / N                     # distribution; exact 1/8, 1/4, 5/8
```

```
## [1] 3.12442
## [1] 1.16601
## ms
##       1       2       4
## 0.12502 0.25026 0.62472
```

Recipe 3: conditional probability by filtering

```
# Two dice. Estimate P(at least one six | sum >= 9). Exact: 7/10.
set.seed(7)
N = 100000
d1 = sample(1:6, N, replace = TRUE)     # vectorized: all N runs at once
d2 = sample(1:6, N, replace = TRUE)
B = (d1 + d2 >= 9)
A = (d1 == 6 | d2 == 6)
mean(A[B])
```

```
## [1] 0.7034222
```

Recipe 4: "keep going until..." (W25 Q5b)

```
# Draw cards without replacement until the first spade; return the number of
cards.
cards_until_spade = function() {
  deck = sample(rep(c(0, 1), c(39, 13)))   # shuffle: 1 = spade
  return(which(deck == 1)[1])              # position of the first spade
}
set.seed(8)
sims = replicate(100000, cards_until_spade())
mean(sims > 5)            # W25 Q1d: exact 0.22
mean(sims)                # expected number of cards: exact 53/14 = 3.79
```

```
## [1] 0.21913
## [1] 3.76706
```

Shuffling the whole deck once, then finding the first 1, is often simpler than drawing one card at a time. The W25 exam instead updates the probability after each draw. Make sure you can follow both styles.

Recipe 5: box-model sums

```
box = c(-4, -4, -4, -4, -4, -4, 1, 2, 3, 4, 5, 6)   # Review 15 Q1 box
set.seed(9)
sums = replicate(100000, sum(sample(box, 8, replace = TRUE)))
mean(sums); sd(sums) * sqrt((100000 - 1) / 100000)
8 * mean(box); sqrt(8) * sqrt(mean(box^2) - mean(box)^2)      # formulas
```

```
## [1] -2.00818
## [1] 11.159
## [1] -2
## [1] 11.14301
```

Drills

6.1 ★ WRITE

Estimate P(sum of two dice = 7) with 100,000 runs, using a `for` loop that appends to a results vector.

6.2 ★★ WRITE

Let M be the maximum of two dice. Write `simMax()` and code that estimates E(M) and SE(M). Then compute both **exactly** in R, using `outer(1:6, 1:6, pmax)` to list all 36 outcomes.

6.3 ★★ FILL

This function draws **without replacement** from a box with `n0` zeros and `n1` ones until it has drawn **two** 1's, and returns the number of draws. Fill in the blanks. Assume `n1 >= 2`.

```
draw_two_ones = function(n0, n1) {
  ones_seen = 0
  draws = 0
  while (________) {
    p = ________                # chance the next draw is a 1
    draws = draws + 1
    if (runif(1) < p) {
      ones_seen = ________
      n1 = ________
    } else {
      n0 = ________
    }
  }
  return(________)
}
```

6.4 ★★ WRITE

Monty Hall (Lecture 10). Simulate one game where you always **switch**, returning 1 if you win the car. Estimate P(win | switch). (Hint: you win by switching exactly when your first pick was wrong.)

6.5 ★★ WRITE

Birthday problem (Lecture 10). Estimate P(at least two of 23 people share a birthday), with 365 equally likely days. Use `any(duplicated(v))` or `length(unique(v)) < 23`. Then compute the exact answer in one line with `prod`.

6.6 ★★ EXPLAIN

For each block, say in words exactly what is being estimated.

```
# (a)
x = replicate(10000, sum(sample(1:6, 3, replace = TRUE)))
mean(x >= 15)
# (b)
y = replicate(10000, max(sample(c(1, 2, 2, 5), 2)))
sd(y) * sqrt(9999 / 10000)
# (c)
h = sample(0:1, 10000, replace = TRUE); r = sample(1:6, 10000, replace = TRUE)
mean(h[r == 6])
# (d)
w = replicate(10000, sum(sample(c(1, -1), 100, replace = TRUE, prob = c(18, 20) /
38)))
mean(w > 0)
```

6.7 ★★★ WRITE

Two tickets are drawn **without** replacement from {0, 4, 4, 8}. X is the first and Y the second. Estimate Cov(X, Y) by simulation (exact: −8/3, Lecture 13). Then change one argument to draw *with* replacement and check that the covariance is about 0.

6.8 ★★★ WRITE

You bet $1 on a single roulette number 100 times. Each bet wins $35 with probability 1/38 and loses $1 otherwise. (a) Estimate P(ahead after 100 bets) by simulation. (b) Compute it exactly with `pbinom`. (Hint: if W is the number of wins, net = 36W − 100.)

6.9 ★★★ WRITE

Law of averages (Lecture 14). For n = 100, 1,000, 10,000 and 100,000 fair tosses, simulate once and print |heads − n/2| and |percent heads − 50%|. Predict the trend before you run it, and connect it to SE(count) = $\sqrt{n}/2$ and SE(percent) = $50\%/\sqrt{n}$.

6.10 ★★★ WRITE

Expected number of **distinct** faces in 6 rolls of a die. Estimate it, and compare with the exact value $6\,(1-(5/6)^6)$. (Why is that the exact value? Hint: E(sum of indicators).)

MODULE 7

The Gauntlet: timed, exam conditions

Goal: prove the R points are yours · **Time:** about 35 min

Rules: 35 minutes, pen and paper, no R, no notes. Then check every answer in R and against the key. Target: 9 of 10 fully correct. If you miss that, redo Module 4's table and Module 6's recipes, and try again tomorrow.

G1 ★★ TRANSLATE

A box has 9 red and 11 blue balls. (a) Draw 6 without replacement: R command for P(at least 3 red). (b) Draw 6 *with* replacement: R command for P(exactly 3 red). (c) Same as (b): P(fewer than 3 red).

G2 ★★ TRANSLATE

A basketball team wins each game with probability 0.6, independently, over 80 games. (a) Exact P(wins between 45 and 55, inclusive). (b) The normal approximation to (a) with continuity correction, written with `pnorm` and all numbers filled in.

G3 ★★ WRITE

Write `simX()` for this experiment: roll a die and get N. Then draw N tickets *with* replacement from the box {0, 1, 1}. X is the number of 1's drawn. Then write code that estimates E(X) (exact 7/3) and SE(X) from 10,000 runs.

G4 ★★ FILL

Fill in the blanks so the code estimates P(the first ace appears at or before the 5th card) when you deal from a shuffled deck.

```
first_ace = function() {
  deck = sample(rep(c(0, 1), c(____, ____)))
  return(which(________)[1])
}
sims = numeric(50000)
for (i in ________) {
  sims[i] = ________
}
prob = ________
```

G5 ★★ EXPLAIN

What does the code estimate? Give the exact value too.

```
z = replicate(20000, {
  t = sample(c(1, 2, 2, 3), 2)
  t[1] * (t[2] - 1)
})
mean(z == 0)
```

G6 ★★ DEBUG

This is supposed to estimate the SE of the number of heads in 10 fair tosses (exact $\sqrt{2.5} = 1.58$). Find the **three** bugs.

```
heads = numeric(10000)
for (i in 1:10000) {
  heads = sum(sample(c(0, 1), 10))
}
se = sd(heads) * sqrt(10000 / 9999)
```

G7 ★★ WRITE

A diagnostic test (Lecture 10, Example 2): 1 in 1,000 employees uses drugs, P(+ | user) = 0.98 and P(+ | non-user) = 0.01. Write a simulation of 1,000,000 employees, fully vectorized with no loop, that estimates P(user | +). The exact value is 98/1097 ≈ 0.089.

G8 ★★ WRITE

Write `cointoss(p)` two more ways: (a) with `sample` and no `runif`; (b) as a single expression using a logical comparison.

G9 ★★★ EXPLAIN

What box does this function simulate? What are E(X) and SE(X)?

```
sx = function(n) {
  p = runif(n)
  return(-3*(p < 1/3) + 1*(p >= 1/3 & p < 5/6) + 10*(p >= 5/6))
}
```

G10 ★★★ WRITE

Review 15 Q10: you draw 4 coins without replacement from 3 pennies, 2 nickels, 4 dimes and 1 quarter. Explain why the class formula SE = $\sqrt{n} \times$ SD of box doesn't apply. Then write the simulation that estimates the SE of the total, and compare with the formula's (wrong) value.

MODULE 8

Flashcards: memorize until instant

Goal: zero hesitation on the syntax · **Time:** about 20 min

Cover the right column of the key's flashcard page, or just write the code here and check. Go through all 40 until you get every one right in a row. Do one round again the morning of the exam.

#	Task	Your code
1	Vector of 5 zeros	
2	The box {1, 1, 2, 4}	
3	39 zeros then 13 ones	
4	0, 0.25, 0.5, 0.75, 1	
5	Last element of v	
6	v without its first element	
7	Elements of v above 3	
8	How many elements of v equal 2	
9	Fraction of v that is positive	
10	Positions where v equals 1	
11	First position where v equals 1	
12	Is 5 one of the values in v?	
13	10 mod 3	
14	5!	
15	10 choose 3	
16	Ordered choices of 3 from 10	
17	$P(X = 4)$, $X \sim \text{Bin}(10, 0.3)$	
18	$P(X \le 4)$, $X \sim \text{Bin}(10, 0.3)$	
19	$P(X \ge 4)$, $X \sim \text{Bin}(10, 0.3)$	
20	$P(X > 4)$, $X \sim \text{Bin}(10, 0.3)$	
21	$P(2 \le X \le 5)$, $X \sim \text{Bin}(10, 0.3)$	
22	P(exactly 2 red) drawing 5 w/o replacement from 7 red, 13 blue	
23	P(at most 2 red), same box and draws	
24	Area left of $z = 1.5$	
25	$P(X > 80)$, X normal with mean 70, SD 8	
26	z for the 95th percentile	
27	One uniform number on (0, 1)	
28	1 with probability p, else 0	
29	One die roll	
30	Ten die rolls	

31	Shuffle the vector v	
32	3 tickets w/o replacement from box	
33	Biased coin: 1 w.p. 0.3, 5 tosses	
34	Function header with argument n	
35	Loop i from 1 to N	
36	Loop while x is below 10	
37	Append x to the vector res	
38	Estimate SE from N simulated values in sims	
39	Estimate P(A | B) from logical vectors A and B	
40	Run expression 1000 times, keep results	